



UNIVERSITÉ DE ROUEN

ANNÉE UNIVERSITAIRE 2024–2025

Rapport de projet de fin d'année

*De la photo au catalogue : une application de scan intelligente
pour livres*

ScanLib

**Reconnaissance de tranches de livres pour gestion de
bibliothèque intelligente**

Auteurs : Flora BENANE / Andrianina ANDRIANTSIALONINA / Luka MEUNIER

Encadrant : Maxime BERAR

Formation : Master 1 – SIME

Université : UFR Sciences et Techniques

19 juin 2025

Table des matières

Liste des figures	3
Glossaire	4
1 Introduction	6
1.1 Contexte général	6
1.2 Objectifs du projet	6
1.3 Portée de l'application	6
1.4 Méthodologie générale	6
1.5 Fonctionnalités clés attendues	7
1.6 Technologies utilisées	7
1.7 Défis techniques identifiés	7
2 Étude de l'existant / État de l'art	8
2.1 Applications similaires	8
2.2 Limites et opportunités	8
3 Analyse des besoins	10
3.1 Besoins fonctionnels	10
3.2 Besoins non fonctionnels	10
3.3 Diagramme des cas d'utilisation	11
4 Choix technologiques	13
4.1 Langage de programmation	13
4.2 Bibliothèques et Interface de Programmation d'Applications (API)s utilisées	13
4.3 Architecture	13
4.4 Justification des choix	13
5 Conception	14
5.1 Modélisation UML	14
5.2 Maquettes / wireframes	16
5.3 Structure de la base de données	17
6 Implémentation	18
6.1 Organisation des fichiers	18
6.2 Chaîne de traitement d'un scan	18
6.3 Algorithmes clés	20
6.4 Interface utilisateur	21
7 Entraînement et intégration du modèle YOLOv8	22
7.1 Présentation du modèle YOLOv8	22
7.2 Préparation du jeu de données	22
7.3 Phase d'entraînement	22
7.4 Évaluation du modèle	23
7.5 Conversion et optimisation pour mobile	23
7.6 Intégration dans l'application Android	24

8 Résultats obtenus	26
8.1 Fonctionnalités implémentées	26
8.2 Démonstration de l'application	26
9 Limites et pistes d'amélioration	31
9.1 Fonctionnalités non abouties	31
9.2 Évolutions possibles	31
10 Conclusion et bilan du projet	32
10.1 Résumé des apports	32
10.2 Limites rencontrées et perspectives	32
Annexes	33
Sitographie	34

Table des figures

1	Diagramme global des cas d'utilisation principaux de ScanLib.	11
2	Répartition des cas d'utilisation entre l'utilisateur et l'application.	12
3	Diagramme de classes de la couche persistance de ScanLib.	14
4	Diagramme de séquence : ajout d'un livre (scan, enrichissement, insertion).	14
5	Diagramme de séquence : recherche et filtrage dans la collection locale.	15
6	Diagramme de séquence global : échanges entre modules fonctionnels.	15
7	Diagramme d'activités : workflow global du traitement d'un livre scanné.	15
8	Wireframe des principales interfaces de l'application : accueil, scan, et détails.	16
9	Maquettes des principales interfaces de l'application : accueil, scan, et détails.	16

Glossaire

Android Système d'exploitation mobile développé par Google, utilisé pour les smartphones et tablettes. [7](#)

API Google Books Service web permettant d'accéder aux métadonnées de livres référencés par Google Books. [6](#), [7](#), [9–11](#), [32](#)

Babelio Plateforme communautaire francophone dédiée à la gestion, au partage et à l'évaluation de livres. [8](#)

Book Collector Application de gestion de livres offrant des fonctions avancées de tri, de visualisation et d'import de métadonnées, compatible Android, iOS et Web. [8](#)

BookScanner Application permettant la reconnaissance de couverture et l'extraction de texte pour cataloguer des livres, principalement via la couverture. [8](#)

CameraX Bibliothèque Jetpack proposée par Google pour simplifier et unifier l'accès à la caméra sur Android, offrant une API moderne et flexible pour la capture d'images et de vidéos. [7](#)

Dagger Hilt Bibliothèque facilitant l'injection de dépendances et l'architecture modulaire dans les applications Android. [7](#)

Figma Outil de conception et de prototypage collaboratif d'interfaces utilisateur pour le web et le mobile. [7](#)

Git Système de gestion de versions distribué permettant de suivre l'évolution du code source et de faciliter le travail collaboratif. [6](#), [7](#)

GitHub Plateforme d'hébergement et de gestion de dépôts Git. [6](#), [7](#)

Goodreads Plateforme communautaire de partage d'avis, d'évaluations et de gestion de collections de livres. [8](#)

Google Lens Application de reconnaissance d'images et de texte basée sur l'intelligence artificielle, proposant une fonction OCR générale. [8](#)

Google ML Kit Ensemble d'API de machine learning proposé par Google pour intégrer la reconnaissance optique de caractères (OCR) et d'autres fonctionnalités IA dans les applications mobiles. [7](#), [32](#)

Kotlin Langage de programmation moderne, concis et sécurisé, principalement utilisé pour le développement Android. [7](#)

Libib Application mobile et web permettant de gérer une bibliothèque personnelle via le scan de codes-barres ISBN, la gestion de listes, le prêt d'ouvrages et la synchronisation en ligne. [8](#)

MyLibrary Application mobile gratuite centrée sur le scan de codes-barres pour gérer et rechercher des collections de livres. [8](#)

Room Bibliothèque d'abstraction de base de données SQLite pour Android. [6](#), [7](#), [9](#), [32](#)

SCRUM Méthode agile de gestion de projet basée sur des itérations courtes et des réunions régulières. [6](#)

Slack Plateforme collaborative de messagerie instantanée pour équipes. [6](#)

SQLite Moteur de base de données relationnelle embarqué, utilisé pour la persistance locale dans les applications mobiles. [7](#), [9](#), [11](#)

TensorFlow Lite Version légère de la bibliothèque TensorFlow, optimisée pour l'exécution de modèles d'apprentissage automatique sur appareils mobiles. [7](#)

Trello Outil de gestion visuelle de tâches en mode Kanban. [6](#)

YOLOv8 Modèle avancé de vision par ordinateur pour la détection et la segmentation d'objets en temps réel. [7](#), [32](#)

Acronymes

API Interface de Programmation d'Applications. [1](#), [13](#), [17–19](#), [21](#)

DAO Data Access Object. [13](#), [21](#)

ISBN International Standard Book Number. [8](#)

MVVM Model-View-ViewModel. [6](#), [11](#), [13](#), [18](#)

NER Named Entity Recognition. [32](#)

OCR Reconnaissance Optique de Caractères. [6–10](#), [13](#), [26](#), [32](#)

UI Interface Utilisateur. [6](#), [10](#), [11](#)

1 Introduction

1.1 Contexte général

La gestion efficace d'une bibliothèque représente aujourd'hui un défi majeur pour les institutions ainsi que pour les particuliers passionnés de lecture. Référencer manuellement chaque ouvrage, localiser un titre spécifique ou éviter l'achat involontaire de doublons constitue souvent un processus fastidieux, nécessitant une automatisation appropriée pour simplifier l'inventaire. Bien que certaines solutions mobiles facilitent l'identification d'un livre grâce au scan d'un code-barres, peu d'entre elles permettent la reconnaissance simultanée de plusieurs ouvrages à partir d'une unique photographie des tranches. Le projet ScanLib se positionne ainsi comme une application mobile innovante, apportant une solution pratique à ces problématiques en facilitant la gestion, la recherche et l'identification automatisée des livres via la reconnaissance visuelle de leurs tranches.

1.2 Objectifs du projet

L'objectif principal de ce projet est de développer une solution mobile capable de scanner la tranche d'un livre, d'identifier automatiquement le titre et l'auteur par reconnaissance optique, puis d'enrichir ces données à l'aide de sources en ligne. L'application doit également offrir des fonctionnalités de gestion locale (ajout, suppression, modification), de recherche avancée et de filtrage des livres présents dans la collection.

1.3 Portée de l'application

L'application couvre l'ensemble du cycle de gestion d'une bibliothèque personnelle :

- Détection et extraction des titres et auteurs depuis une photo ;
- Recherche et récupération d'informations enrichies (image, éditeur, résumé, etc.) via l'[API Google Books](#) ;
- Sauvegarde locale des ouvrages dans une base [Room](#) ;
- Consultation, filtrage et modification de la base ;
- Gestion de groupes de favoris et personnalisation de la bibliothèque.

La version actuelle se concentre sur les fonctionnalités essentielles, tout en posant les bases pour de futures évolutions (par exemple, **reconnaissance de couverture** ou *détection en temps réel*).

1.4 Méthodologie générale

Le projet a été mené selon la méthode [SCRUM](#), avec une répartition du travail en sprints hebdomadaires, des points journaliers pour synchroniser l'équipe via [Slack](#) et une démarche itérative privilégiant l'incrémentation des fonctionnalités. Chaque membre s'est vu attribuer des tâches précises via [Trello](#), et la gestion des versions a été assurée grâce à [Git](#) et [GitHub](#). Le projet a ainsi été découpé en plusieurs sprints, chacun visant à livrer un incrément fonctionnel :

- Initialisation du projet et définition du cahier des charges ;
- Conception de l'architecture logicielle (modèle [Model-View-ViewModel \(MVVM\)](#), choix des technologies) ;
- Développement des modules principaux ([Reconnaissance Optique de Caractères \(OCR\)](#), base de données, [Interface Utilisateur \(UI\)](#)) ;

- Intégration des fonctionnalités avancées (groupes, favoris, filtres) ;
- Tests, validation et itérations selon les retours.

Cette méthodologie a permis d'assurer une progression continue du projet, tout en facilitant la collaboration et la gestion des priorités.

1.5 Fonctionnalités clés attendues

Les fonctionnalités attendues et livrées incluent :

- Scan de la tranche du livre et extraction automatique des informations via [OCR](#) ;
- Recherche enrichie d'ouvrages via [API Google Books](#) ;
- Gestion locale de la bibliothèque (ajout, suppression, modification, filtrage, etc.) ;
- Interface utilisateur intuitive et accessible ;
- Synchronisation partielle entre les données locales et les sources externes.

1.6 Technologies utilisées

Le projet **ScanLib** mobilise un ensemble cohérent de technologies avancées pour garantir performance, modularité et facilité d'utilisation :

- **Application mobile** : développement natif sous [Android](#) en [Kotlin](#).
- **Vision par ordinateur** :
 - *Segmentation des images* : [YOLOv8](#).
 - *Reconnaissance optique de caractères (OCR)* : [Google ML Kit](#).
- **Recherche d'informations** : [API Google Books](#) pour enrichir automatiquement les métadonnées des ouvrages.
- **Base de données locale** : [Room](#) ([SQLite](#)) pour assurer la persistance efficace des données.

En complément, plusieurs outils et frameworks facilitent l'intégration technique et le développement collaboratif :

- [TensorFlow Lite](#) pour optimiser la reconnaissance de texte sur mobile.
- [Dagger Hilt](#) pour l'injection de dépendances et l'architecture modulaire de l'application.
- [Git/GitHub](#) pour la gestion collaborative du code source.
- [Figma](#) pour la conception ergonomique et moderne des interfaces utilisateur.

1.7 Défis techniques identifiés

Plusieurs défis majeurs se sont imposés au cours du projet :

- Reconnaissance optique fiable sur des tranches abîmées ou peu lisibles ;
- Identification précise de l'édition exacte d'un ouvrage, les différences d'édition pouvant se manifester par des variations de texte ou des logos sur la tranche ;
- Difficulté à distinguer, sur la tranche, le nom du livre de celui de l'auteur, les deux pouvant apparaître dans des ordres et graphismes variés ;
- Correspondance entre le texte détecté et la bonne édition dans [API Google Books](#) ;
- Synchronisation cohérente entre la base locale et les données distantes ;
- Maintien de la performance et de la réactivité sur terminaux mobiles ;
- Prise en main rapide de technologies récentes ([Room](#), [CameraX](#), etc.) ;
- Robustesse face aux erreurs utilisateurs ou aux cas particuliers (doublons, scans partiels, etc.).

2 Étude de l'existant / État de l'art

L'étude de l'existant a été menée dans le but d'analyser les solutions actuelles de gestion de bibliothèque, qu'elles soient issues du marché ou de la littérature scientifique. Cette démarche a permis d'identifier leurs principales fonctionnalités, d'en évaluer les atouts et les limites, et ainsi de dégager les leviers d'innovation sur lesquels repose la spécificité du projet **ScanLib**.

2.1 Applications similaires

2.1.1 Gestion de bibliothèque personnelle

Il existe aujourd'hui de nombreuses applications mobiles et web permettant de gérer une collection de livres :

- **Libib** (Android/iOS/Web) : permet de scanner les codes-barres [International Standard Book Number \(ISBN\)](#) pour ajouter des livres, gérer des listes, prêter des ouvrages et synchroniser ses données en ligne ;
- **Book Collector** (Android/iOS/Web) : offre des fonctions avancées de gestion, de tri et de visualisation, avec un accent mis sur la numérisation par code-barres et l'import de métadonnées via des bases tierces ;
- **MyLibrary** (Android/iOS) : application gratuite et intuitive centrée sur le scan de codes-barres pour l'ajout de livres, la gestion de collections et la recherche rapide.

2.1.2 Reconnaissance optique (OCR) et catalogage

Peu d'applications proposent la reconnaissance optique directe de la tranche du livre.

- **Google Lens** : propose une reconnaissance **OCR** très générale, mais sans workflow dédié à la gestion de bibliothèque ;
- **BookScanner** : application qui combine reconnaissance de couverture et extraction de texte, mais qui se focalise surtout sur la couverture et pas sur la tranche, avec une efficacité variable selon la qualité des photos.

2.1.3 Gestion de collections spécialisées

Des solutions telles que [Goodreads](#) ou [Babelio](#) sont orientées communauté et partage d'avis, mais l'ajout d'un livre reste basé sur la recherche manuelle ou le scan de code-barres.

2.2 Limites et opportunités

2.2.1 Limites identifiées

L'analyse des solutions existantes a mis en évidence plusieurs limitations récurrentes :

- **Dépendance aux codes-barres** : la plupart des applications reposent sur la lecture du code-barres ([ISBN](#)) pour identifier les ouvrages. Ce choix exclut de nombreux livres anciens, importés ou auto-édités, qui n'en possèdent pas ou dont le code est dégradé ;
- **Négligence de la tranche** : très peu d'outils exploitent la tranche, pourtant souvent la seule partie visible d'un livre en bibliothèque. Or, l'analyse de la tranche

présente des défis techniques spécifiques, tels que le texte vertical, les polices atypiques ;

- **Recherche textuelle rudimentaire** : le processus de correspondance entre texte détecté et données issues de bases en ligne reste souvent limité à une simple recherche par mots-clés, entraînant des doublons ou des erreurs ;
- **Interfaces peu ergonomiques** : de nombreuses applications présentent des interfaces vieillissantes, surchargées ou peu intuitives, peu adaptées aux usages mobiles contemporains ;
- **Fonctionnalités de personnalisation restreintes** : les options de filtrage, de tri ou de regroupement sont souvent rigides ou insuffisamment paramétrables pour répondre à des besoins avancés.

2.2.2 Opportunités exploitées par ScanLib

Le projet **ScanLib** se fonde précisément sur ces constats pour proposer une série d'innovations ciblées :

- **OCR spécialisé sur la tranche** : une chaîne de traitement d'image a été conçue pour détecter efficacement le texte vertical sur tranches de livres, tout en gérant les artefacts visuels et la faible lisibilité ;
- **Algorithmes de matching enrichis** : l'approche de recherche combine plusieurs champs (titre, auteur) pour interroger des bases comme [API Google Books](#), limitant ainsi les erreurs d'association et maximisant la pertinence des résultats ;
- **Expérience utilisateur modernisée** : l'interface, épurée et réactive, met l'accent sur l'accessibilité des fonctions clés, la clarté de l'affichage, et la personnalisation (groupes, favoris, tri) ;
- **Gestion locale flexible** : grâce à l'intégration de [Room \(SQLite\)](#), l'utilisateur conserve le contrôle total sur sa bibliothèque, même hors ligne, avec des possibilités d'ajout, de modification, de suppression et de regroupement des ouvrages ;
- **Perspectives d'extension via l'IA** : l'architecture du projet ouvre la voie à des développements futurs, tels que la reconnaissance automatique de couvertures par vision par ordinateur, l'identification d'éditeurs par logo, ou l'analyse statistique des lectures.

L'état de l'art met en évidence les limites des solutions fondées sur le code-barres ou la recherche textuelle simple. **ScanLib** propose une alternative innovante, centrée sur la reconnaissance des tranches, le croisement intelligent de données, et une gestion locale souple et personnalisée.

3 Analyse des besoins

3.1 Besoins fonctionnels

Les fonctionnalités décrites ci-dessous définissent le périmètre du projet **ScanLib**.

1. **Scan et extraction automatique des métadonnées** : L'application doit permettre, via la caméra du smartphone, de scanner la tranche d'un livre. Un module de reconnaissance optique de caractères (**OCR**) est intégré pour automatiser l'extraction du titre et de l'auteur.
2. **Recherche dans une base externe** : Une fois les métadonnées extraites, l'application interroge une base de données en ligne (**API Google Books**) afin d'identifier l'ouvrage correspondant.
3. **Enrichissement des informations** : Outre le titre et l'auteur, l'application doit enrichir la fiche du livre avec des éléments complémentaires (couverture, résumé, éditeur, date de parution, etc.), afin d'améliorer la lisibilité et la valeur ajoutée de la bibliothèque numérique personnelle.
4. **Gestion locale complète des ouvrages** : L'utilisateur peut :
 - ajouter un livre (par scan ou manuellement) ;
 - modifier des informations (corrections, ajouts) ;
 - supprimer un ouvrage.
5. **Recherche et filtrage avancés** : Pour faciliter la gestion d'une grande collection, des fonctionnalités de recherche et de filtrage sont prévues :
 - par genre (roman, essai, bande dessinée, etc.) ;
 - par score du livre ;
 - par année de publication ou d'ajout ;
 - par mot-clé (auteur, éditeur).
6. **Organisation par groupes et favoris** : L'utilisateur doit pouvoir créer ses propres groupes (ex : «À lire») et marquer des livres comme favoris, pour une organisation personnalisée.

3.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité du système et influencent directement l'expérience utilisateur ainsi que la robustesse de l'application.

1. **Performance et fluidité** : L'application doit rester fluide, même sur des appareils aux performances modestes. Les traitements d'image, l'**OCR** et les requêtes réseau doivent être optimisés.
2. **Interface utilisateur ergonomique et accessible** : L'interface (**UI**) doit être claire, intuitive et conforme aux standards de design mobile pour une prise en main immédiate.
3. **Robustesse face aux erreurs et cas limites** : L'application doit rester stable face à des entrées incomplètes, des erreurs **OCR** ou des réponses partielles de l'**API Google Books**, grâce à une gestion des exceptions et des messages d'erreur explicites.
4. **Modularité et évolutivité** : L'architecture de l'application doit permettre l'ajout de nouvelles fonctionnalités (ex. nouveaux filtres, synchronisation cloud) selon une

organisation modulaire du code et une documentation claire, s'appuyant sur le modèle [MVVM](#).

3.3 Diagramme des cas d'utilisation

Un diagramme UML des cas d'utilisation a été élaboré afin de représenter les interactions principales entre les différents acteurs et composants du système ScanLib.

Acteurs principaux :

- **Utilisateur** : acteur central, initiateur de toutes les actions (scan, ajout, modification, suppression, recherche, gestion des groupes, etc.) ;
- **Scanner (OCR)** : module dédié à l'extraction automatisée de texte à partir de la tranche du livre ;
- **Base externe** : service [API Google Books](#) interrogé pour enrichir et valider les données extraites ;
- **Base locale** : système de persistance Room/[SQLite](#) pour la gestion et la sauvegarde des livres et des groupes ;
- **Interface utilisateur (UI)** : couche d'interaction offrant à l'utilisateur l'accès à toutes les fonctionnalités du système.

Cas d'utilisation représentatifs :

- **Ajouter un livre** (via scan ou saisie manuelle) ;
- **Modifier** ou compléter les informations d'un livre ;
- **Supprimer** un livre ;
- **Rechercher** un livre par mot-clé ;
- **Filtrer** la collection selon des critères (genre, année, note, groupe, etc.) ;
- **Gérer les groupes/favoris** (création, modification, suppression, attribution de livres à des groupes).

Le diagramme ci-dessous synthétise les principaux cas d'utilisation ainsi que les interactions majeures entre l'utilisateur, les modules fonctionnels (OCR, API, gestion des groupes) et la base locale.

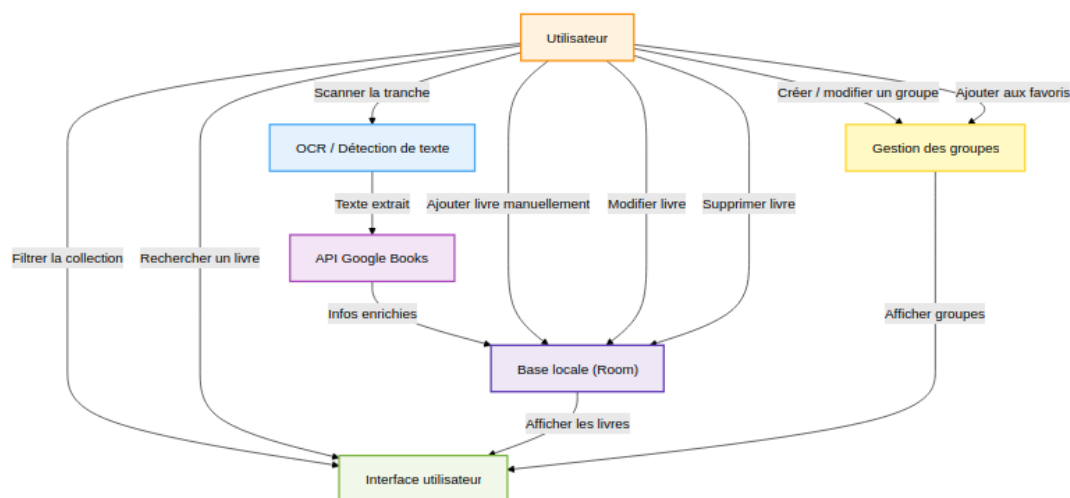


FIGURE 1 – Diagramme global des cas d'utilisation principaux de ScanLib.

Le diagramme suivant présente la répartition des responsabilités entre les deux grands acteurs du système : l'utilisateur et l'application elle-même. Chaque action utilisateur correspond à un cas d'utilisation précis, tandis que certaines fonctionnalités sont entièrement gérées par l'application en arrière-plan.

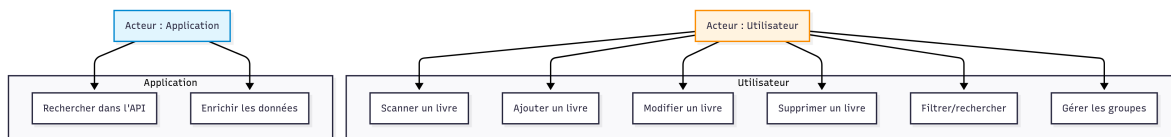


FIGURE 2 – Répartition des cas d'utilisation entre l'utilisateur et l'application.

4 Choix technologiques

4.1 Langage de programmation

Le choix de **Kotlin** s'est imposé naturellement pour ce projet, car il s'agit du langage privilégié dans le développement Android moderne, mais aussi car il a été présenté et utilisé tout au long du cursus. Kotlin, officiellement recommandé par Google, offre une syntaxe concise et expressive, tout en restant pleinement compatible avec l'écosystème Java existant. Il facilite également l'utilisation des fonctionnalités avancées telles que les coroutines, qui simplifient la gestion des tâches asynchrones, notamment lors des appels réseau ou du traitement d'images.

4.2 Bibliothèques et **APIs** utilisées

Pour la vision par ordinateur, nous avons retenu le modèle **YOLOv8**, reconnu pour sa rapidité et sa précision dans la détection d'objets, même sur des appareils mobiles. La reconnaissance de texte (**OCR**) est assurée par **Google ML Kit**, une bibliothèque maintenue par Google qui fournit des modèles performants optimisés pour Android et utilisables hors ligne. La récupération des métadonnées des livres repose sur l'**API Google Books**, qui donne accès à une vaste base de données de livres et permet d'enrichir automatiquement la fiche de chaque ouvrage détecté. Enfin, pour la persistance des données, nous avons choisi **Room**, une bibliothèque de stockage qui s'appuie sur le concept de **Data Access Object (DAO)** pour proposer une interface simple et robuste au-dessus de la base SQLite embarquée.

4.3 Architecture

Le modèle architectural **MVVM** (Model-View-ViewModel) s'est également imposé car il a été largement abordé et appliqué lors des séances de cours. Ce choix s'inscrit dans la continuité des bonnes pratiques enseignées, tout en facilitant la séparation des responsabilités, la réutilisation des composants et la maintenance de l'application. L'injection de dépendances est assurée par **Hilt**, une bibliothèque issue de Dagger, qui permet de gérer efficacement la création et le cycle de vie des objets au sein de l'application. Cela garantit un code plus modulaire, facilement testable et évolutif.

4.4 Justification des choix

L'ensemble de ces choix technologiques reflète à la fois une volonté de suivre les standards industriels, mais aussi de s'appuyer sur les technologies et architectures étudiées dans le cadre du cursus. Ainsi, l'intégration native à l'écosystème Android (Kotlin, Room, Hilt) et le recours à des **APIs** gratuites, bien documentées et reconnues, assurent à la fois robustesse, évolutivité et cohérence pédagogique. L'utilisation d'outils open-source performants (YOLOv8 pour la détection, ML Kit pour l'OCR) permet d'atteindre une haute précision tout en restant indépendant de solutions propriétaires. Enfin, l'adoption d'une architecture modulaire et maintenable garantit la pérennité du projet et en facilite l'évolution future.

5 Conception

5.1 Modélisation UML

La modélisation UML constitue une étape clé dans la conception logicielle pour la maintenabilité de l'architecture à long terme. Les schémas qui suivent traduisent les choix d'architecture (le pattern MVVM) et la structuration des modules essentiels.

- **Diagramme de classes** : structure logique des entités persistées (livres, groupes, DAO, repository) ;
- **Diagrammes de séquence** : illustration des principaux scénarios utilisateur (ajout, recherche, gestion) ;
- **Diagramme d'activités** : synthèse du workflow fonctionnel de l'ajout d'un livre.

Diagramme de classes de la base de données Ce diagramme détaille la structure de la base Room : il met en évidence la relation plusieurs-à-plusieurs entre les entités Book et FavoriteGroup via la table de jointure, ainsi que la séparation des responsabilités.

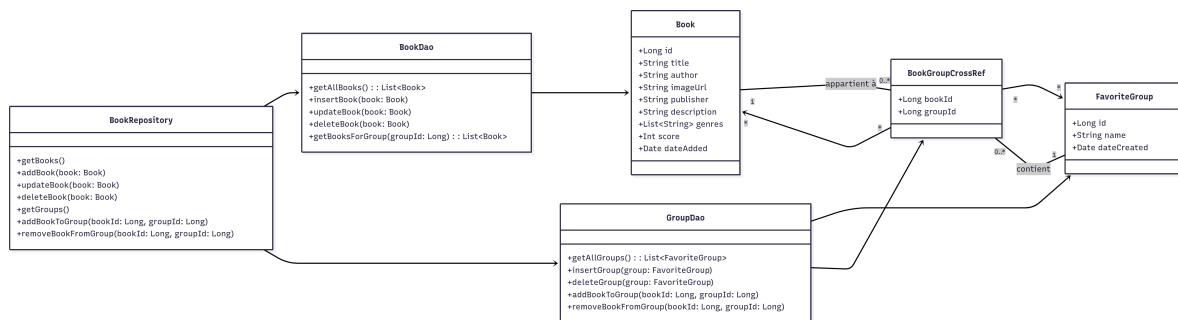


FIGURE 3 – Diagramme de classes de la couche persistance de ScanLib.

Diagramme de séquence – Ajout d'un livre Ce diagramme illustre le scénario complet de l'ajout d'un livre : du scan de la tranche à l'insertion en base locale, en passant par la validation utilisateur et l'enrichissement via l'API externe.

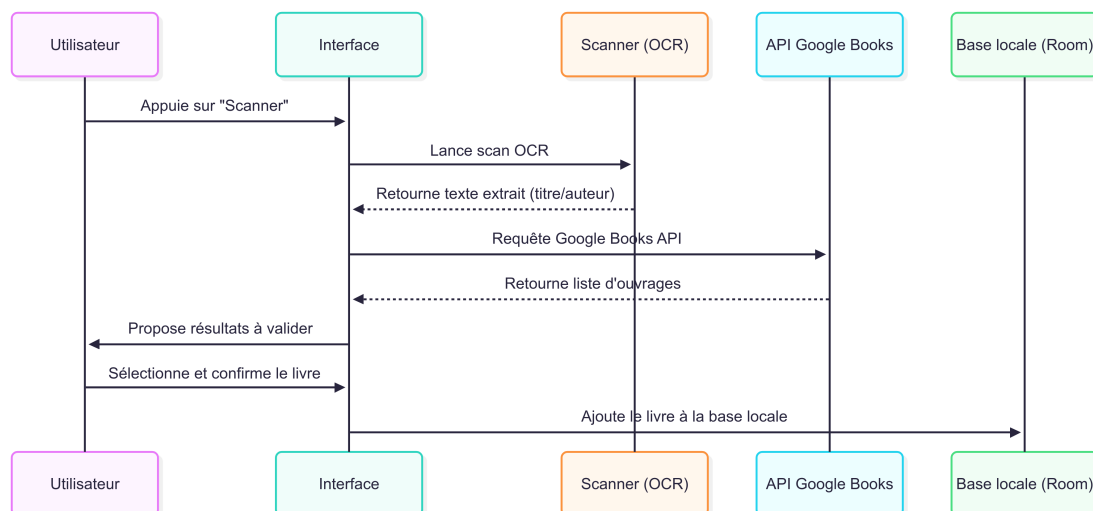


FIGURE 4 – Diagramme de séquence : ajout d'un livre (scan, enrichissement, insertion).

Diagramme de séquence – Recherche et filtrage Ce diagramme présente les échanges intervenant lors de la recherche ou du filtrage de livres dans la collection locale.

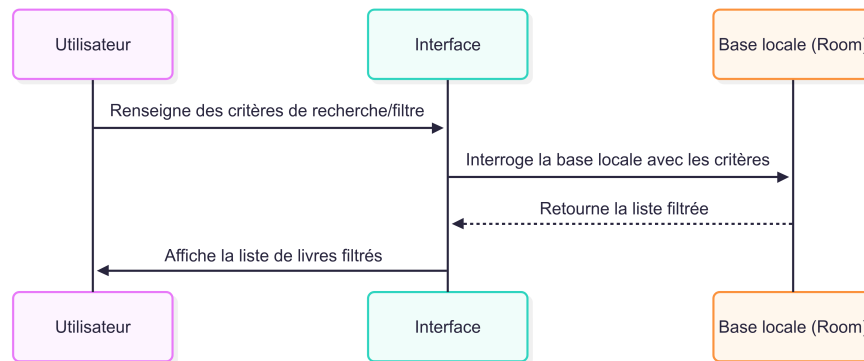


FIGURE 5 – Diagramme de séquence : recherche et filtrage dans la collection locale.

Diagramme de séquence global Ce diagramme donne une vision d'ensemble des échanges entre les modules fonctionnels, du scan à l'organisation par groupes.

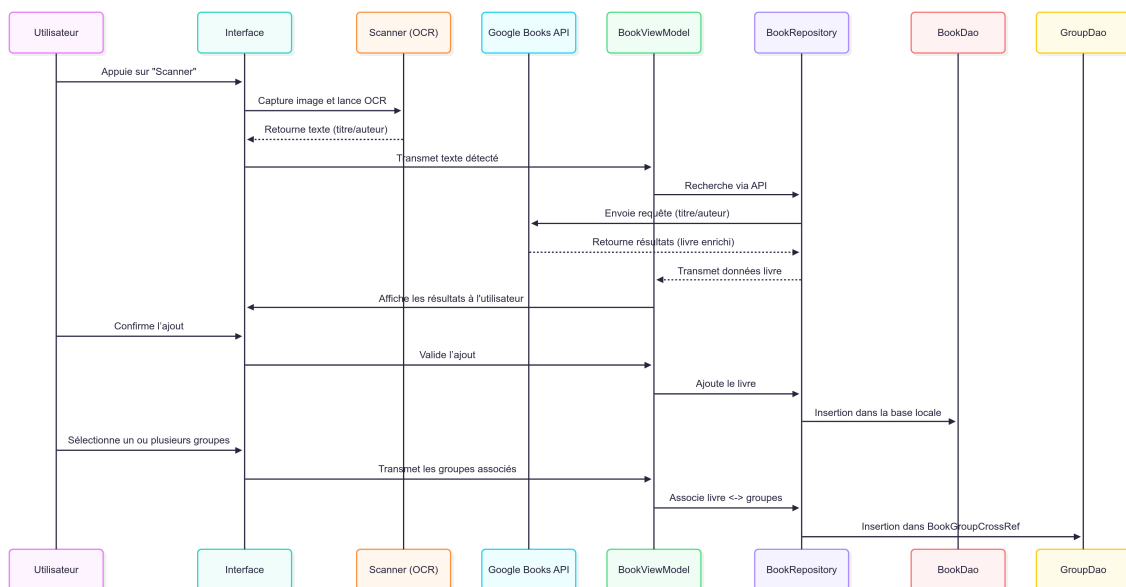


FIGURE 6 – Diagramme de séquence global : échanges entre modules fonctionnels.

Diagramme d'activités – Workflow global Le diagramme d'activités synthétise les étapes du traitement d'un livre, depuis la capture jusqu'à la sauvegarde, en intégrant les alternatives (erreur, absence de correspondance).

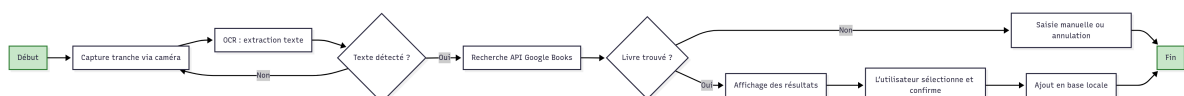


FIGURE 7 – Diagramme d'activités : workflow global du traitement d'un livre scanné.

L'ensemble de ces schémas a servi de référence tout au long du développement.

5.2 Maquettes / wireframes

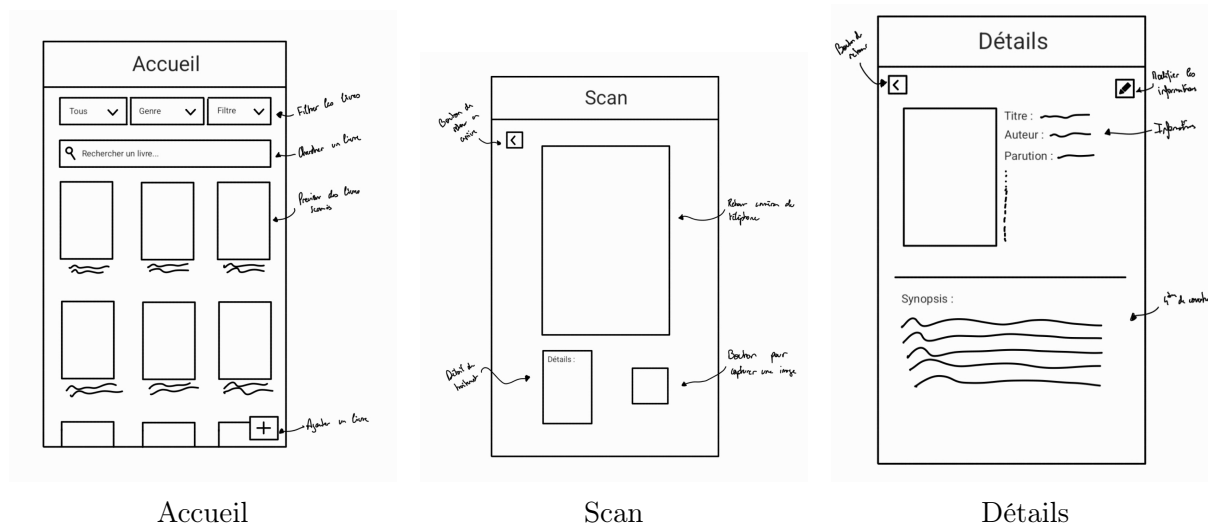


FIGURE 8 – Wireframe des principales interfaces de l'application : accueil, scan, et détails.

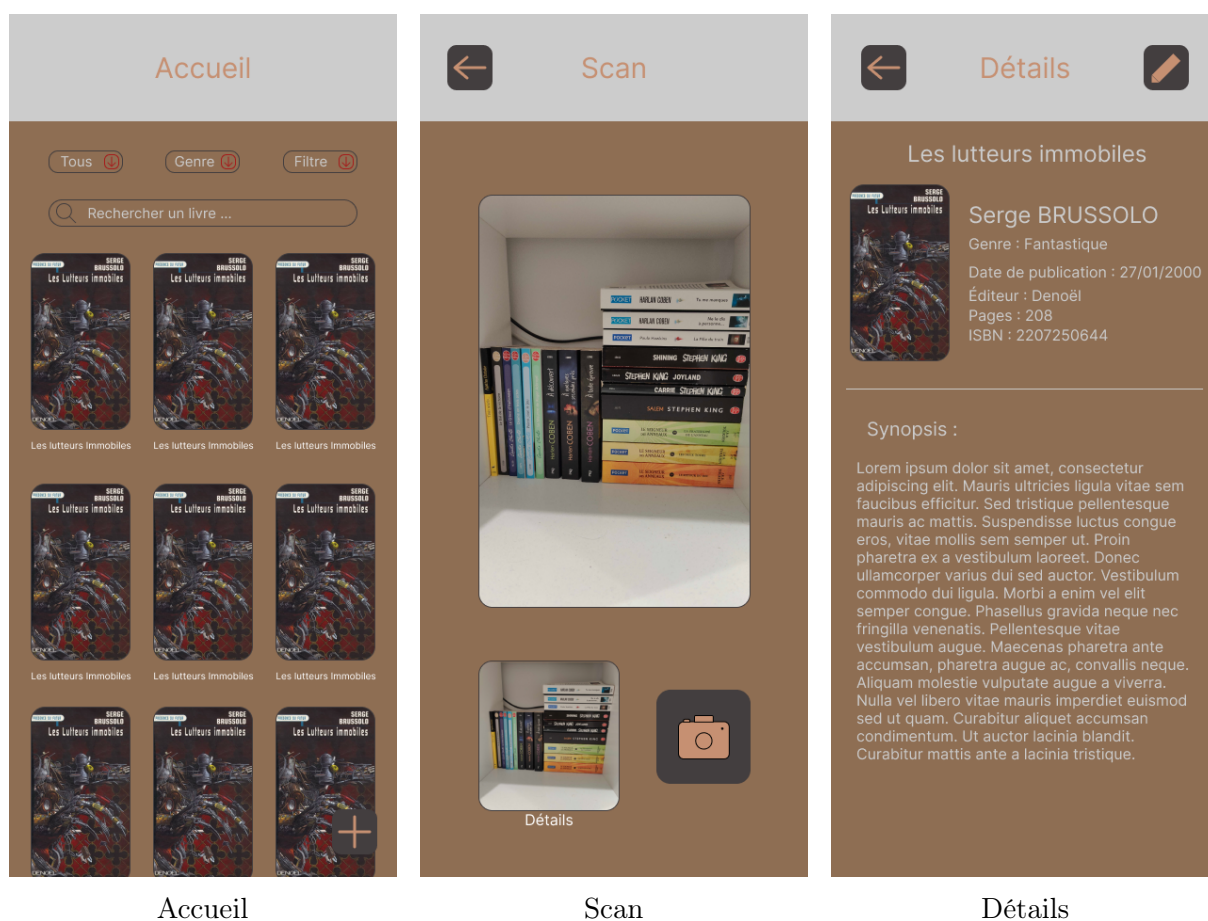


FIGURE 9 – Maquettes des principales interfaces de l'application : accueil, scan, et détails.

5.3 Structure de la base de données

- Book : id, titre, auteurs, éditeur, année, image, description, catégories, note, etc.
- BookDao : accès CRUD et filtres.
- BookRepository : gestion des appels [API](#) et synchronisation.

Listing 1 – Structure de l'entité Book

```
@Entity(tableName = "books")
@Parcelize
data class Book(
    @PrimaryKey val id: String,
    val title: String,
    val authors: List<String>,
    val publisher: String?,
    val publishedDate: String?,
    val description: String?,
    val pageCount: Int,
    val industryIdentifiers: List<String>?,
    val categories: List<String>?,
    val averageRating: Double?,
    val ratingsCount: Int?,
    val thumbnailUrl: String?,
    val smallThumbnailUrl: String?,
    val previewLink: String?,
    val infoLink: String?,
    val country: String?,
    val textSnippet: String?
) : Parcelable
```

6 Implémentation

6.1 Organisation des fichiers

Le projet respecte une architecture [MVVM](#) (Model-View-ViewModel), séparant clairement les responsabilités dans l'application. L'organisation des fichiers principaux est décrite ci-dessous.

1. **app/**
Contient le point d'entrée de l'application :
 - `ScanLibApp.kt` : classe d'initialisation, configuration des modules Hilt.
2. **data/**
Représente la couche «Model» de l'application. Elle gère les entités, la base de données locale et les appels aux services externes :
 - `Book.kt`, `GroupBook.kt` : entités représentant les livres.
 - `BookDao`, `GroupBookDao`, `BookDatabase` : gestion de la base Room.
 - `BookRepository` : interface entre base de données et logique applicative.
 - `GoogleBooksService.kt` : interaction avec l'[API](#) Google Books.
 - `BookSyncResult.kt`, `ScanResult.kt`, `FilterState.kt` : objets intermédiaires de traitement.
3. **ml/**
Regroupe les traitements liés à la vision par ordinateur :
 - `BookSpineDetector.kt` : encapsule le modèle YOLOv8 pour la détection des tranches.
 - `BookSpineOCR.kt` : effectue la reconnaissance de texte via Google ML Kit.
4. **ui/**
Contient toute l'interface utilisateur, découpée en trois sous-parties :
 - **adapter/** : adaptateurs pour RecyclerView :
 - `BookAdapter.kt`, `ScanResultAdapter.kt`, `BookGroupSelectAdapter.kt`, etc.
 - **view.fragments/** : contient les fragments de navigation représentant les écrans de l'application :
 - `HomeFragment`, `ScanFragment`, `ProcessingFragment`, `ScanResultFragment`, etc.
 - **viewmodel/** : logiques de présentation (couche ViewModel) associées à chaque fragment :
 - `BookViewModel`, `HomeViewModel`, `DetailsViewModel`, `ScanViewModel`.

Cette structuration modulaire favorise la lisibilité du code, l'indépendance des composants, et la facilité de maintenance ou de test.

6.2 Chaîne de traitement d'un scan

Le traitement complet d'un scan se déroule en plusieurs étapes successives, depuis la capture ou l'import d'image jusqu'à la sauvegarde sélective des livres reconnus dans la base locale. L'orchestration principale est assurée dans `ProcessingFragment`, avec l'appui des classes métier dédiées à la détection (YOLOv8), à l'OCR et aux requêtes API.

1. Capture ou import d'image

L'image est capturée via `CameraX` dans `ScanFragment`, ou bien importée depuis la galerie via `HomeFragment`. Dans les deux cas, l'image est transmise à `ProcessingFragment`.

2. Chargement de l'image et affichage initial

Dans `ProcessingFragment`, l'image est chargée avec `BitmapFactory` et affichée à l'utilisateur (`imageOriginal`) avant traitement.

3. Détection des tranches de livres (YOLOv8)

La classe `BookSpineDetector` applique un modèle YOLOv8 converti en TensorFlow Lite. Elle effectue :

- un redimensionnement de l'image à 640×640 ,
- la normalisation des pixels en float,
- l'inférence du modèle,
- un post-traitement avec suppression non maximale (NMS) pour filtrer les détections.

Le résultat est une liste de régions (`RectF`) correspondant aux tranches détectées.

4. Reconnaissance de texte (OCR)

Pour chaque boîte détectée, `BookSpineOCR` effectue les opérations suivantes :

- conversion de la boîte vers les dimensions originales,
- extraction d'un sous-bitmap avec zoom $\times 6$,
- reconnaissance de texte via Google ML Kit,
- nettoyage du texte pour obtenir une ligne formatée (titre + auteur).

5. Affichage des textes OCR à l'utilisateur

Les lignes extraites sont affichées dans `ProcessingFragment` sous forme de liste. Le bouton "Continuer" devient actif uniquement si au moins une ligne est non vide.

6. Requête vers Google Books API

En appuyant sur "Continuer", l'application appelle `syncBooksFromValTexts()` dans `BookViewModel`. Celui-ci délègue à `BookRepository`, qui :

- interroge l'API Google Books pour chaque ligne OCR,
- parse la réponse JSON pour créer des objets `Book`,
- collecte les livres trouvés et ceux non trouvés.

7. Affichage des livres trouvés

Les livres identifiés sont transmis au `ScanResultFragment`, où ils sont présentés à l'utilisateur sous forme de liste cliquable. Celui-ci peut :

- consulter les détails de chaque livre,
- sélectionner manuellement ceux qu'il souhaite enregistrer.

Les livres non cochés sont ignorés à l'étape suivante.

8. Sauvegarde dans la base Room

Lorsque l'utilisateur valide via le bouton "Enregistrer", seuls les livres sélectionnés sont sauvegardés localement. L'appel à `insertBook()` insère les objets dans la base Room, en gérant les doublons par la clé primaire.

Ce pipeline permet d'identifier automatiquement les livres à partir d'une photo, tout en laissant à l'utilisateur le choix final des données à enregistrer.

6.3 Algorithmes clés

Cette section détaille les principaux traitements algorithmiques mis en œuvre dans ScanLib, depuis la détection des tranches via YOLOv8 jusqu’à l’insertion des livres détectés dans la base locale.

1. Détection d’objets (YOLOv8)

La détection des tranches de livres repose sur un modèle YOLOv8 converti en TensorFlow Lite. L’image est redimensionnée à 640×640 , normalisée, puis injectée dans le modèle. Un post-traitement est appliqué pour extraire les boîtes pertinentes.

Listing 2 – Détection de tranches (YOLOv8)

```
fun detect(bitmap: Bitmap): Pair<List<RectF>, Size> {
    val resized = bitmap.scale(inputSize, inputSize)
    val inputBuffer = ... // Normalisation RGB
    interpreter.run(inputBuffer, output) // Inference
    val boxes = postProcess(output) // NMS
    return Pair(boxes, Size(inputSize, inputSize))
}
```

2. Suppression non maximale (NMS)

Pour éviter les doublons, une NMS filtre les boîtes trop proches (avec IoU élevé) en ne gardant que les plus probables. Cette étape améliore la précision.

Listing 3 – Suppression non maximale (NMS)

```
private fun nonMaxSuppression(...): List<RectF> {
    return boxes.zip(scores)
        .sortedByDescending { it.second }
        .filter { box -> picked.all { computeIoU(box, it) < seuil } }
        .map { it.first }
}
```

3. Reconnaissance de texte (OCR)

Chaque boîte détectée est extraite de l’image source, agrandie, puis analysée par Google ML Kit. Le texte est ensuite nettoyé pour faciliter les recherches.

Listing 4 – Extraction OCR avec nettoyage

```
val inputImage = InputImage.fromBitmap(zoomedImage, 0)
val resultText = recognizer.process(inputImage).await().text
val line = cleanTextSingleLine(resultText) // Suppression de chiffres isolés,
normalisation
```

4. Nettoyage du texte OCR

Le texte brut est converti en une ligne lisible : suppression des lignes numériques, caractères parasites, et espaces superflus.

Listing 5 – Nettoyage OCR simplifié

```
fun cleanTextSingleLine(rawText: String): String {
    return rawText.lines()
        .filter { it.isNotBlank() && !it.matches(Regex("^\\d+$")) }
        .joinToString(" ")
        .replace(Regex("\\s+"), " ")
        .trim()
}
```

5. Recherche Google Books

Pour enrichir les résultats OCR, une requête HTTP est envoyée à l'[API](#) Google Books, avec parsing JSON à la volée. Un seul résultat est extrait par requête.

Listing 6 – Recherche enrichie via API Google Books

```
val url = "https://www.googleapis.com/books/v1/volumes?q=$query"
val response = client.newCall(request).execute()
val volumeInfo = jsonObject["items"]?.firstOrNull()?.asJsonObject?.get("volumeInfo")
```

6. Sauvegarde dans Room (DAO)

Les livres sélectionnés par l'utilisateur sont insérés localement via un [DAO](#), avec remplacement automatique en cas de doublon (REPLACE).

Listing 7 – Insertion des livres dans Room

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insertBooks(books: List<Book>)
```

Chaque algorithme a été choisi pour son efficacité mobile et sa simplicité d'intégration. Ensemble, ils forment une chaîne robuste de traitement intelligent d'image à données enrichies.

6.4 Interface utilisateur

L'interface utilisateur de l'application s'organise autour de plusieurs écrans principaux, conçus pour guider l'utilisateur tout au long du processus de gestion de sa bibliothèque. L'écran d'accueil affiche l'ensemble des livres enregistrés sous forme de grille et propose des outils de recherche et de filtrage dynamiques. Un écran spécifique permet de réaliser un scan en prenant une photo, tandis qu'un autre écran est dédié à l'import d'images depuis la galerie du téléphone. L'utilisateur peut également accéder à un écran d'ajout manuel pour saisir les informations d'un livre qui ne serait pas détecté automatiquement. Après chaque opération de scan ou d'import, un écran de résultats présente les livres identifiés, avec la possibilité de consulter ou de modifier les détails de chaque ouvrage. L'interface inclut également un mode de sélection multiple pour supprimer facilement plusieurs livres à la fois, ainsi que des boîtes de dialogue qui facilitent l'application de filtres ou la validation d'actions importantes. L'ensemble est pensé pour offrir une expérience fluide et adaptée à un usage mobile.

7 Entraînement et intégration du modèle YOLOv8

7.1 Présentation du modèle YOLOv8

YOLO (You Only Look Once) est une famille de modèles de détection d'objets conçus pour allier rapidité et précision en une seule passe sur l'image. La version YOLOv8, proposée par Ultralytics, améliore les performances précédentes grâce à une architecture allégée, une meilleure précision, et une compatibilité native avec les formats ONNX et TFLite, facilitant son déploiement sur mobile.

YOLOv8 utilise un pipeline unifié pour prédire les boîtes englobantes et les scores de confiance, avec une architecture optimisée pour la détection d'objets. Sa précision et sa compacité en font un modèle bien adapté aux contraintes des applications mobiles telles que ScanLib.

Dans notre cas d'usage, le modèle est utilisé pour détecter les tranches de livres visibles sur une photographie d'étagère. Chaque tranche détectée est ensuite transmise à un module d'OCR pour en extraire le texte. L'objectif est d'isoler des régions pertinentes avec un minimum de faux positifs, dans un contexte de fond visuel souvent chargé (livres côte à côte, variations de couleurs, titres partiellement visibles, etc.).

Nous avons retenu l'architecture YOLOv8n (nano), la plus légère de la gamme, afin de garantir des performances satisfaisantes sur les appareils Android.

7.2 Préparation du jeu de données

Pour entraîner le modèle YOLOv8 à détecter les tranches de livres sur des photos d'étagères, nous avons utilisé un jeu de données disponible publiquement sur la plateforme **Roboflow**, intitulé : *Book Spine Detection*¹.

Ce dataset comporte au total **9 879 images** annotées selon le format YOLOv5/YOLOv8, avec une seule classe d'objet : **book spine**. Chaque annotation contient les coordonnées normalisées des boîtes englobantes, compatibles avec les formats d'entrée de YOLO.

La répartition des données a été réalisée comme suit :

- **Entraînement (train)** : 8 717 images
- **Validation (val)** : 782 images
- **Test (test)** : 380 images

Les images proviennent de différentes bibliothèques et étagères, incluant des angles variés, des tranches partiellement visibles ou floues, ce qui permet de rendre l'entraînement plus robuste aux conditions réelles d'utilisation sur mobile.

7.3 Phase d'entraînement

L'entraînement du modèle a été réalisé à l'aide de la bibliothèque **Ultralytics** sous Python, dans un environnement Google Colab avec accélération GPU (*Tesla T4*). Le modèle sélectionné est **YOLOv8n**, une version compacte optimisée pour les systèmes mobiles, qui présente un bon équilibre entre taille, vitesse et précision.

- **Outil utilisé** : API Ultralytics (`yolov8`) exécutée via un notebook Python. Le modèle a été chargé avec `YOLO("yolov8n.pt")` et entraîné grâce à la commande `model.train()`.
- **Paramètres d'entraînement** :

1. <https://universe.roboflow.com/koteitan/book-spine-detection-2cci9/dataset/1>

- `epochs = 20` : nombre d'itérations complètes sur le jeu d'entraînement,
- `imgsz = 640` : dimension des images d'entrée redimensionnées,
- `batch` : paramètre par défaut (automatiquement ajusté selon la mémoire disponible),
- `data = "data.yaml"` : fichier de configuration du dataset exporté depuis Roboflow.
- **Structure du fichier `data.yaml`** : ce fichier contient les chemins vers les images annotées pour chaque sous-ensemble :
 - `train : .../train/images`, avec 8717 images,
 - `val : .../valid/images`, avec 782 images,
 - `test : .../test/images`, avec 380 images.
 Il définit également le nombre de classes (`nc = 1`) et le nom de la classe cible (`names = ["book_spine"]`).
- **Jeu de données utilisé** : comme présenté précédemment, nous avons utilisé le dataset *Book Spine Detection* issu de la plateforme Roboflow², déjà structuré au format YOLOv8 et accompagné d'un fichier `data.yaml` décrivant les chemins, les classes et les répartitions des données.
- **Durée d'entraînement** : environ 25 minutes avec GPU, incluant les phases de prétraitement, d'augmentation automatique (recadrage, rotation, bruit) et d'optimisation du modèle.

L'ensemble du processus a permis de générer un modèle performant prêt à être évalué, puis exporté en tflite pour une utilisation embarquée.

7.4 Évaluation du modèle

Une fois l'entraînement terminé, le modèle YOLOv8n a été évalué sur l'ensemble de validation pour mesurer sa capacité de généralisation. Les résultats sont exprimés selon les métriques classiques en détection d'objets.

- **mAP@0.5** : 96.9 % — indique qu'en moyenne, les boîtes détectées recouvrent les vraies tranches à plus de 50 % dans 97 % des cas. Il s'agit d'une très bonne précision sur des critères « tolérants ».
- **mAP@0.5 :0.95** : 81.4 % — mesure plus stricte, calculée comme la moyenne des mAP pour des seuils d'IoU allant de 0.5 à 0.95. Ce score confirme que le modèle reste performant même avec des exigences de cadrage élevées.
- **Précision** : 94.6 % — parmi les boîtes détectées, 94.6 % sont correctes.
- **Rappel** : 90.2 % — le modèle retrouve 90.2 % des tranches réellement présentes.

Analyse des erreurs Certaines erreurs subsistent :

- Détection manquée sur des tranches floues ou très étroites ;
- Fusions incorrectes entre deux livres collés sans séparation visible ;
- Faux positifs sur des objets verticaux (bordures, cadres).

7.5 Conversion et optimisation pour mobile

Pour permettre l'inférence directement sur appareil Android, le modèle YOLOv8 entraîné (au format `.pt`) doit être converti en format **TFLite**, compatible avec TensorFlow Lite. La conversion s'est déroulée en trois étapes principales.

2. <https://universe.roboflow.com/koteitan/book-spine-detection-2cci9>

1. Export du modèle au format ONNX Nous avons d'abord exporté le modèle entraîné (YOLOv8n) en `.onnx` à l'aide de la fonction intégrée à Ultralytics :

Listing 8 – Export YOLOv8 vers ONNX

```
from ultralytics import YOLO

model = YOLO("best.pt")
model.export(format="onnx")
```

Cette commande génère un fichier `best.onnx`, utilisé comme point de départ pour la conversion.

2. Conversion ONNX → TensorFlow La conversion du modèle ONNX vers un graphe TensorFlow est effectuée grâce à l'outil `onnx2tf`. Il suffit d'exécuter la commande suivante dans le terminal :

Listing 9 – Conversion ONNX vers TensorFlow

```
onnx2tf -i best.onnx
```

Cela produit un dossier `saved_model/` contenant une version TensorFlow du modèle, exploitable par `TFLiteConverter`.

3. Conversion TensorFlow → TFLite Enfin, nous utilisons TensorFlow Lite pour produire un modèle léger destiné à l'intégration mobile :

Listing 10 – Conversion vers TFLite

```
import tensorflow as tf

converter = tf.lite.TFLiteConverter.from_saved_model("saved_model")
tflite_model = converter.convert()

with open("best.tflite", "wb") as f:
    f.write(tflite_model)
```

Résultat final Le modèle obtenu `best.tflite` est compact (11.6 MB dans notre cas) et peut être directement intégré dans l'application Android.

7.6 Intégration dans l'application Android

Le modèle quantifié (`.tflite`) est embarqué dans le dossier `assets/` et chargé à l'exécution via un `MappedByteBuffer`, assurant un accès rapide et sans copie mémoire :

Listing 11 – Chargement du modèle TFLite

```
val file = assetManager.openFd("best-v1.tflite")
val buffer = FileInputStream(file.fileDescriptor).channel.map(
    FileChannel.MapMode.READ_ONLY, file.startOffset, file.declaredLength
)
val interpreter = Interpreter(buffer)
```

L'image capturée ou importée est redimensionnée à 640×640 , puis transformée en `ByteBuffer` avec normalisation RGB avant inférence :

Listing 12 – Prétraitement

```
val resized = bitmap.scale(640, 640)
val inputBuffer = ByteBuffer.allocateDirect(...).order(ByteOrder.nativeOrder()
    )
// Remplissage RGB [0,1]...
```

Après inférence :

Listing 13 – Exécution et post-traitement

```
interpreter.run(inputBuffer, output)
val results = postProcess(output) // seuil, NMS, etc.
```

Cette méthode `detect()` est appelée depuis `ProcessingFragment`, déclenchant l'ensemble du pipeline. Les boîtes détectées sont ensuite transmises à l'OCR pour extraction de texte. L'ensemble du traitement est effectué localement, assurant fluidité et réactivité sur mobile.

8 Résultats obtenus

8.1 Fonctionnalités implémentées

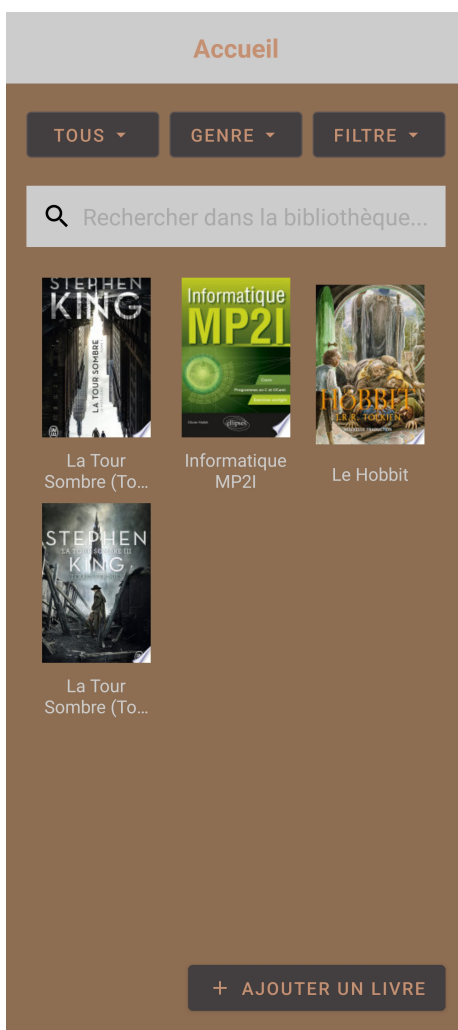
- Scan photo ou import,
- Détection IA des tranches,
- [OCR](#) pour titres/auteurs,
- Recherche Google Books,
- Gestion (ajout, modif, suppression) des livres,
- Filtres dynamiques et recherche,
- Affichage détaillé et édition,
- Base Room.

8.2 Démonstration de l'application

La démonstration suivante illustre le parcours type d'un utilisateur pour ajouter un livre à sa bibliothèque via l'application.

8.2.1 Accueil

L'utilisateur arrive sur l'écran d'accueil (*HomeFragment*), qui présente la liste de ses livres et offre un bouton pour lancer le scan.



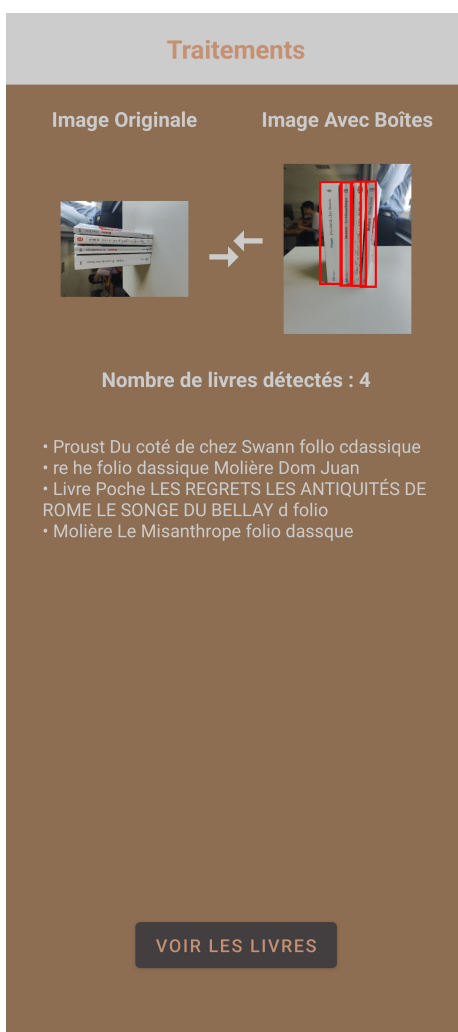
8.2.2 Accès à la fonction Scan

En appuyant sur le bouton dédié, l'utilisateur navigue vers l'interface de capture de photo (*ScanFragment*), où il peut cadrer une étagère ou une pile de livres à scanner.



8.2.3 Prise de photo et traitement

Après avoir pris la photo, l'application affiche un écran de traitement indiquant la progression de la détection des livres et de la reconnaissance de texte.



8.2.4 Affichage des résultats

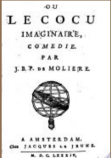
L'utilisateur consulte ensuite l'écran de résultats, qui affiche les livres détectés sur l'image, avec possibilité de sélectionner ceux à ajouter à sa bibliothèque.

Liste des livres trouvés



A la Recherche du Temps Perdu 1: du Côté de Chez Swann (Première Partie)
Marcel Proust
Éditeur inconnu

☐



Sganarelle ou Le cocu imaginaire,
Molière
Éditeur inconnu

☐



Français, Première. L'œuvre et son parcours : Baudelaire, Les Fleurs du Mal, parcours "Alchimie poétique : la boue et l'or"
Rémy Arcemisbèhère, Morgan Trouillet
Editions Ellipses

☐



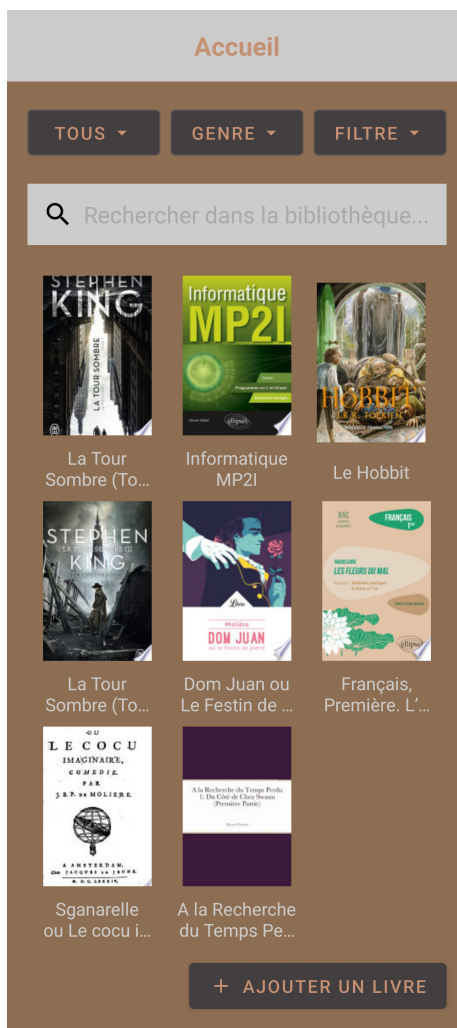
Dom Juan ou Le Festin de pierre
Molière
J'ai Lu

☐

ENREGISTRER

8.2.5 Retour à l'accueil et recherche

Une fois la sélection validée, l'utilisateur revient sur l'écran d'accueil, où les nouveaux livres apparaissent automatiquement. Il peut ensuite utiliser la barre de recherche pour filtrer sa collection et accéder rapidement à un ouvrage particulier.



9 Limites et pistes d'amélioration

9.1 Fonctionnalités non abouties

Bien que l'application remplisse déjà ses objectifs principaux, certaines fonctionnalités attendues dans le cahier des charges restent à perfectionner ou n'ont pas pu être implémentées à ce stade du développement.

- **Pas de scan en temps réel.**

Le cahier des charges initial prévoyait la détection des tranches de livres et l'OCR en temps réel, directement sur le flux vidéo de la caméra. Actuellement, l'application se limite à l'analyse d'images fixes capturées par l'utilisateur. Cette absence de traitement en continu réduit l'ergonomie et la rapidité du scan, et empêche une utilisation fluide et instantanée, comme celle d'un scanner de QR code.

- **OCR imparfait.**

Le système de reconnaissance de texte donne parfois des résultats approximatifs, notamment en cas de texte de petite taille, de mauvaise orientation, ou de tranche abîmée. Les erreurs nécessitent alors une correction manuelle, ce qui peut ralentir l'utilisation.

- **Pas de gestion des utilisateurs.**

L'application ne propose pas de système de comptes ou de profils utilisateurs. Toute la bibliothèque de livres est donc considérée comme appartenant à une seule personne. Cela limite les usages collaboratifs ou familiaux, et empêche la séparation de collections ou la personnalisation des données selon l'utilisateur.

9.2 Évolutions possibles

Afin de rendre l'application plus complète et adaptée à des usages variés, plusieurs axes d'amélioration peuvent être envisagés pour enrichir les fonctionnalités et renforcer la robustesse du système.

- **Mode vidéo temps réel.**

Intégrer la détection de livres et l'OCR directement sur le flux vidéo de la caméra permettrait un scan plus rapide et intuitif, à la manière des applications modernes de reconnaissance visuelle.

- **Reconnaissance d'entités nommées (NER).**

Ajouter une étape de traitement automatique pour identifier les noms d'auteur, de titre ou d'éditeur dans le texte extrait fiabiliserait et structurerait l'import des données dans la base.

- **Sauvegarde cloud.**

Offrir une synchronisation avec un espace cloud (Google Drive, Dropbox, etc.) garantirait la sécurité des données utilisateur et permettrait de retrouver sa bibliothèque sur plusieurs appareils.

- **API supplémentaires (OpenLibrary, BNF, etc.).**

Interroger d'autres bases de données (ex. OpenLibrary, BNF) permettrait d'augmenter la couverture des ouvrages reconnus et d'améliorer la qualité des métadonnées récupérées.

- **Prétraitement d'image amélioré.**

Mettre en œuvre des techniques avancées de traitement d'image (redressement, amélioration du contraste, suppression des reflets) pourrait fortement améliorer la qualité de l'OCR et la fiabilité de la détection.

En conclusion, la poursuite du développement selon ces axes permettrait de franchir un cap significatif en termes d’ergonomie, de fiabilité et de richesse fonctionnelle, et de s’adapter à des besoins plus larges.

10 Conclusion et bilan du projet

10.1 Résumé des apports

Le projet **ScanLib** a démontré la faisabilité d’une solution mobile permettant de reconnaître et de gérer automatiquement une bibliothèque personnelle à partir d’une simple photo d’étagère. Grâce à l’intégration de techniques d’intelligence artificielle ([YOLOv8](#) pour la détection, [OCR](#) via [Google ML Kit](#)), d’un enrichissement automatique des métadonnées ([API Google Books](#)) et d’une base locale fiable ([Room](#)), l’application remplit son objectif de simplification et d’automatisation de la gestion de livres.

Les principales fonctionnalités attendues ont été implémentées avec succès :

- Détection des tranches de livres par un modèle léger et optimisé ;
- Extraction automatique des titres et auteurs par [OCR](#) ;
- Enrichissement intelligent des données via une API externe ;
- Interface fluide et adaptée aux usages mobiles ;
- Gestion locale des ouvrages avec filtres et favoris.

10.2 Limites rencontrées et perspectives

Le projet a cependant révélé certaines limites :

- L’absence d’analyse en temps réel sur flux vidéo limite la fluidité d’usage ;
- Le module [OCR](#) reste sensible à la qualité des images (flou, inclinaison, faible contraste) ;
- La distinction automatique entre titre, auteur et éditeur n’est pas encore fiable à 100 % ;
- Aucune gestion multi-utilisateur ni sauvegarde cloud n’a été implémentée.

Ces constats ouvrent la voie à plusieurs axes d’amélioration :

- Ajout d’un traitement vidéo temps réel avec détection continue ;
- Amélioration de la reconnaissance par NER ([Named Entity Recognition \(NER\)](#)) pour mieux structurer les textes OCRisés ;
- Intégration d’autres bases externes (OpenLibrary, BNF) pour augmenter la couverture des données ;
- Synchronisation multi-appareils grâce au cloud.

En résumé, **ScanLib** constitue une base solide et extensible pour des solutions de gestion de bibliothèque automatisée. Il allie efficacité algorithmique, adaptabilité mobile et ouverture vers des extensions futures pertinentes.

Annexes

Exemple de configuration Room

```
@Database(entities = [Book::class], version = 2)
@TypeConverters(BookConverters::class)
abstract class BookDatabase : RoomDatabase() {
    abstract fun bookDao(): BookDao
}
```

Détection de tranches (extrait)

```
val (boxes, modelSize) = detector.detect(bitmap)
```

Requête Google Books (extrait)

```
val book = googleBooksService.searchBook(titleAuthor)
```

Sitographie

<https://developer.android.com/training/data-storage/room?hl=fr>
<https://developer.android.com/develop?hl=fr>
<https://developer.android.com/media/camera/camerax>
https://android.googlesource.com/platform/external/tensorflow/+/ec63214f098a2bfc87b628219ad0718750d4e930/tensorflow/lite/g3doc/guide/get_started.md
<https://developer.android.com/training/dependency-injection/hilt-android>
https://developers.google.com/books/docs/v1/getting_started
<https://www.tothenew.com/blog/building-a-text-recognition-app-using-camerax-and-ml-kit-in-jetpack-compose>
<https://github.com/android/camera-samples>
<https://docs.ultralytics.com>
<https://universe.roboflow.com>
<https://developer.android.com/topic/architecture>